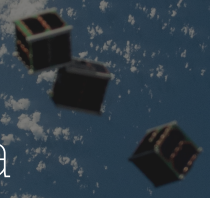
Laboratorio

Bolsa de Palabras Visuales, Detección de Movimiento y Seguimiento de Objetos

Visión por Ordenador I

Ingeniería Matemática



Universidad Pontificia Comillas ICAI

COMILLAS
UNIVERSIDAD PONTIFICIA
ICAI

3º Ingeniería Matemática

Curso 2026/27

Laboratorio

Bolsa de Palabras Visuales, Detección de Movimiento y Seguimiento de Objetos

Profesor:	**Email**
Jaime Boal Martín-Larrauri	jboal@comillas.edu
Lionel Güitta	lglopez@comillas.edu
Daniel Pinilla	dpinilla@comillas.edu
Luis Arias	learias@comillas.edu
José María Bengochea	jmbengochea@comillas.edu
Mario Triviño	mtrivino@comillas.edu

Cover: Tiny satellites, also known as CubeSats, are pictured after being deployed into Earth orbit from a small satellite orbital deployer on the outside of the International Space Station's Kibo laboratory module. Image Credit: NASA/Tracy Dyson

COMILLAS
UNIVERSIDAD PONTIFICIA
ICAI

Contents

1 Sesión 5: Bolsa de Palabras Visuales, Detección de Movimiento y Seguimiento de Objetos **1**
1.1 Materiales . . . 1
1.2 Apartados de la práctica . . . 1
1.3 Observaciones . . . 1
1.4 Qué va a aprender . . . 1
1.5 Evaluación . . . 2

2 Apartado A: Bolsa de palabras **3**
Tarea A.1: Carga de *datasets* de entrenamiento y validación . . . 3
Tarea A.2: Extracción de los descriptores . . . 3
Tarea A.3: Creación del vocabulario . . . 3
Tarea A.4: Entrenamiento del clasificador . . . 3
Tarea A.5: Inferencia en el dataset de entrenamiento . . . 4
Tarea A.6: Inferencia en el dataset de validación . . . 4
Preguntas . . . 4

3 Apartado B: Sustracción de Fondo **5**
Tarea B.1: Carga de vídeo . . . 5
Tarea B.2: Sustracción de fondo mediante diferencia de frames . . . 5
Tarea B.3: Configuración de la Sustracción de Fondo con GMM . . . 5
Tarea B.4: Aplicación de la Sustracción de Fondo . . . 5
Preguntas . . . 6

4 Apartado C: Flujo Óptico **7**
Tarea C.1: Configuración del Flujo Óptico . . . 7
Tarea C.2: Detección de Puntos de Interés . . . 7
Tarea C.3: Cálculo y Visualización del Flujo Óptico . . . 7
Preguntas . . . 7

5 Apartado D: Filtro de Kalman para Seguimiento de Objetos **9**
Tarea D.1: Configuración del Filtro de Kalman . . . 9
Tarea D.2: Predicción y Corrección del Estado . . . 9
Preguntas . . . 10

1

Sesión 5: Bolsa de Palabras Visuales, Detección de Movimiento y Seguimiento de Objetos

1.1. Materiales

En esta práctica se trabajará con los siguientes recursos (puede encontrarlos en la sección de Moodle *Laboratorio/Sesión 4*):

- **`partA.ipynb`**: notebook con el código que deberá desarrollar.
- **`partB-D.ipynb`**: notebook con el código que deberá desarrollar.
- **`data`**: carpeta con el material para trabajar durante la práctica.

1.2. Apartados de la práctica

La Sesión 5 del laboratorio está dividida en los siguientes apartados:

- Apartado A: Bolsa de palabras.
- Apartado B: Sustracción de Fondo.
- Apartado C: Flujo Óptico.
- Apartado D: Filtro de Kalman para Seguimiento de Objetos.

1.3. Observaciones

Aunque el guion de la práctica y los comentarios en Markdown del Notebook estén escritos en español, observe que todo aquello que aparece en las celdas de código está escrito en inglés. Es una buena práctica que todo su código esté escrito en inglés.

Aquellas partes del código que deberá completar están marcadas con la etiqueta **`TODO`**.

Es muy importante que trabaje consultando la documentación de OpenCV[1] para familiarizarse de cara al examen. Tenga en cuenta que en los exámenes no podrá utilizar herramientas de ayuda como Copilot.

1.4. Qué va a aprender

Al finalizar esta práctica, aprenderá a implementar algoritmos de detección de líneas rectas, puntos de interés y a usar de bolsa de palabras visuales. Así como a implementar algoritmos detección de

[1] Documentación de OpenCV: `https://docs.opencv.org/4.x/index.html`

movimiento mediante sustracción de fondo y flujo óptico, y explorará el filtro de Kalman para el seguimiento de objetos.

1.5. Evaluación

La nota que obtenga en esta sesión de laboratorio será la misma que obtenga su pareja. Los apartados de la práctica serán evaluados como refleja la Tabla 1.1.

Tarea	Valor	Resultado
Pregunta A.1	1.5	
Pregunta B.1	1.0	
Pregunta B.2	1.5	
Pregunta C.1	1.5	
Pregunta C.2	1.5	
Pregunta D.1	1.5	
Pregunta D.2	1.5	
Total	**10.0**	

Table 1.1: Valoración de los apartados de la práctica.

2

Apartado A: **Bolsa de palabras**

En este apartado se va a generar una bolsa de palabras a partir de un dataset (descargar de Moodle) de entrenamiento y se evaluará con un dataset de evaluación. Para generar la bolsa de palabras, se seguirán los pasos descritos en las sesiones de teoría:

1. Extracción de descriptores de puntos de interés de las imágenes del dataset.
2. Construcción de diccionario de palabras visuales (K-Means).
3. Para cada imagen del dataset, generar histograma de ocurrencias de palabras del diccionario.
4. Obtener un clasificador utilizando los histogramas de las imágenes del dataset (SVM).

De esta forma, se comenzará con la carga de las imágenes que compondrán los distintos *datasets* para ser procesadas posteriormente.

Tarea A.1: Carga de datasets de entrenamiento y validación

Como ayuda, se le proporciona una clase de Python llamada `Dataset`. Empleando el método `load()` de esta clase, cargue los *datasets* de entrenamiento y validación.

Tarea A.2: Extracción de los descriptores

Una vez cargados los *datasets* se puede proceder a extraer los descriptores de las imágenes que poblarán nuestra bolsa de palabras. Estos descriptores conformarán las ocurrencias de cada característica visual en la bolsa de palabras. Para ello, primero deberá cargar las imágenes del *dataset* de entrenamiento con el `path` de cada una de ellas y hacerlo en escala de grises. Después, empleando los métodos necesarios de `OpenCV` que ha visto durante la sesión, genere los descriptores y guárdelos en una lista.

Tarea A.3: Creación del vocabulario

Con la bolsa de palabras llena con los descriptores, se puede proceder a agrupar los mismos en las distintas palabras del vocabulario que se va a generar. Para ello, se va a ejecutar el algoritmo de clusterización de K-Means que agrupará estos descriptores. En esta tarea debe añadir los descriptores de la lista que obtuvo como resultado en la tarea anterior a la bolsa de palabras `words` que se inicializa en el código y sobre la que ejecutaremos el algoritmo. Fíjese en que se determina el número de palabras deseadas en el vocabulario final con la variable `vocabulary_size` y que esto afectará al desempeño del clasificador más adelante.

Tarea A.4: Entrenamiento del clasificador

Una vez tenemos las características de las imágenes agrupadas en las palabras, se va a entrenar un clasificador que sea capaz de diferenciar entre estos grupos. Este será un modelo basado en una *Support Vector Machine*(SVM). Con este clasificador podremos recibir imágenes externas al *dataset* de

entrenamiento y clasificarlas en nuestro grupo de palabras.

Para entrenar el clasificador se aporta la clase `BoW` como ayuda. Añada los argumentos necesarios a los métodos especificados en la celda de código. Compruebe qué argumentos tiene que añadir accediendo al código de la clase proporcionada y analizando los métodos declarados. El modelo final se guardará en el ordenador para su uso más adelante.

Tarea A.5: Inferencia en el dataset de entrenamiento

Con el modelo entrenado, es hora de ver cómo trabaja y comprobar su desempeño. En este caso la métrica empleada será la precisión y podrá observar la matriz de confusión generada para las predicciones.

Al igual que en la tarea anterior, añada los argumentos necesarios a los métodos especificados en la celda de código.

Tarea A.6: Inferencia en el dataset de validación

Después de extraer los resultados con el dataset de entrenamiento, se tienen que comparar los mismos con los del dataset de validación donde se debería obtener un desempeño inferior.

Al igual que en la tarea anterior, añada los argumentos necesarios.

Preguntas

Pregunta A.1: En busca de los mejores parámetros

Entrene y evalúe el clasificador con vocabularios de 50, 100, 200 y 400 palabras (vocabulary_size). Elija además uno de estos parámetros y estudie su efecto: el kernel del SVM (setKernel() en image_classifier.py), el numero de iteraciones (argumento iterations de train()) o el extractor de caracteristicas (KAZE en lugar de SIFT). Muestre en una tabla la accuracy de entrenamiento y de validacion de cada prueba y justifique que configuracion elegiria.

3

Apartado B: **Sustracción de Fondo**

Tarea B.1: Carga de vídeo

Cargue el vídeo (`'visiontraffic.avi'`) en el cual se detectarán objetos en movimiento. Para ello, realice complete el método `read_video` que puedrá reutilizar en el resto de tareas.

Tarea B.2: Sustracción de fondo mediante diferencia de frames

Realice una sustracción de fondo mediante diferencia de frames, para ello guarde un frame con el fondo estático y úselo como frame de referencia de fondo.

En primer lugar vamos a iterar sobre la lista de frames para mostrar por pantalla frame a frame e iremos avanzando con pulsando la tecla n, para seleccionar el frame de referencia debera pulsar la tecla s.

Posteriormente se realizará la diferencia de frames entre el frame referencia y el frame actual.

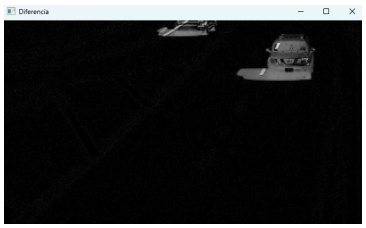


Figure 3.1: Ejemplo de resultado con diferencia de frames.

Tarea B.3: Configuración de la Sustracción de Fondo con GMM

Configure el sustractor de fondo usando el modelo de mezcla de gaussianas adaptativas (MOG2). Para ello, usaremos `cv2.createBackgroundSubtractorMOG2`[1]

Tarea B.4: Aplicación de la Sustracción de Fondo

Aplique la sustracción de fondo en cada frame para extraer los objetos en movimiento.

Visualice el resultado de la detección de movimiento y guarde el vídeo con las imagenes binarias resultantes en la carpeta `data/results`. Utilize el método `cv2.VideoWriter`[2]

[1] Documentación del método MOG2 en OpenCV:
https://docs.opencv.org/4.8.0/d7/d7b/classcv_1_1BackgroundSubtractorMOG2.html

[2] Documentación del método MOG2 en OpenCV:
https://docs.opencv.org/4.8.0/dd/d9e/classcv_1_1VideoWriter.html

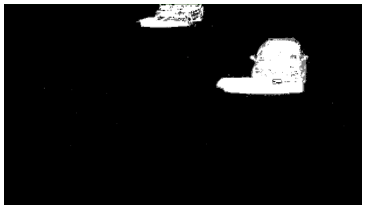


Figure 3.2: Ejemplo de resultado usando MOG2.

Preguntas

Pregunta B.1

¿Cómo afecta la variable `varThreshold` a la precisión de la detección?

Pregunta B.2

¿Qué ventajas presenta `createBackgroundSubtractorMOG2` frente a métodos simples de diferencia de imágenes?

4

Apartado C: **Flujo Óptico**

Tarea C.1: Configuración del Flujo Óptico

Para esta tarea vamos a utilizar el video (`'slow_traffic_small.mp4'`), para cargarlo utilice el método que se ha desarrollado anteriormente. Implemente el flujo óptico de Lucas-Kanade, útil para detectar el movimiento pixel a pixel. Para ello vamos a comenzar por definir los parametros necesarios para utilizar el método `cv2.calcOpticalFlowPyrLK`[1]

Tarea C.2: Detección de Puntos de Interés

Configure los párametros para obtener los puntos de interés iniciales en el primer frame con el método de `cv2.goodFeaturesToTrack`[2].

Tarea C.3: Cálculo y Visualización del Flujo Óptico

Calcule el flujo óptico con el método de Lucas-Kanade aplicando `cv2.calcOpticalFlowPyrLK` y visualice el movimiento de cada punto de interés en los siguientes frames.

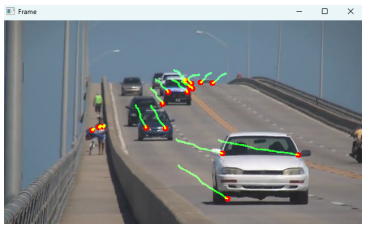


Figure 4.1: Ejemplo de resultado de obtener el flujo ópitco con el metodo de Lucas-Kanade.

Preguntas

Pregunta C.1

¿Qué efecto tiene el parámetro `winSize` en la precisión del nnnnnflujo óptico?

[1] Documentación del método Optical Flow Lucas-Kanade en OpenCV: https://docs.opencv.org/4.8.0/dc/d6b/group__video__track.html#ga473e4b886d0bcc6b65831eb88ed93323

[2] Documentación del método Optical Flow Lucas-Kanade en OpenCV: https://docs.opencv.org/4.8.0/dd/d1a/group__imgproc__feature.html#ga1d6bb77486c8f92d79c8793ad995d541

Pregunta C.2

¿Cómo influye el parámetro `qualityLevel` en la función `cv2.goodFeaturesToTrack` al detectar puntos de interés?

5

Apartado D: **Filtro de Kalman para Seguimiento de Objetos**

Tarea D.1: Configuración del Filtro de Kalman

Para esta tarea vamos a utilizar el video (`'slow_traffic_small.mp4'`), para cargarlo utilice el método que se ha desarrollado anteriormente. Inicialice el filtro de Kalman, para ello haga uso del método `cv2.KalmanFilter`[1] de Opencv, con una matriz de medición y transición adecuada para un seguimiento en dos dimensiones.

Complete el código del bucle que nos permitirá seleccionar el objeto (`cv2.selectROI`)[2] que vamos a seguir. Convierta la región de interés de la imagen a HSV y calcule el histograma `cv2.calcHist`[3] con para el canal con la información de tono (*Hue*).

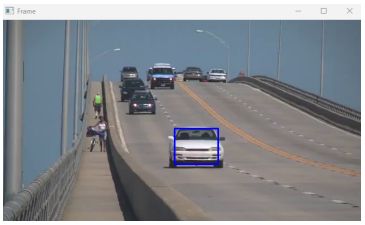


Figure 5.1: Ejemplo de ROI.

Tarea D.2: Predicción y Corrección del Estado

Realice la predicción del estado y corrija la posición estimada en cada iteración.

[1] Documentación del método KalmalFilter en OpenCV:
https://docs.opencv.org/4.8.0/dd/d6a/classcv_1_1KalmanFilter.html
[2] Documentación del método para seleccionar la ROI en OpenCV:
https://docs.opencv.org/4.8.0/d7/dfc/group__highgui.html#gaabb58bae304674b429d9102a7155a86e
[3] Documentación del método para calculo de histograma en OpenCV:
https://docs.opencv.org/4.8.0/d6/dc7/group__imgproc__hist.html#ga4b2b5fd75503ff9e6844cc4dcdaed35d

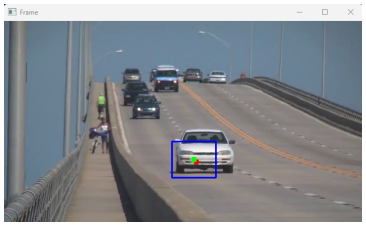


Figure 5.2: Ejemplo de seguimiento con filtro de Kalman.

Preguntas

Pregunta D.1

¿Cómo afecta el valor de `transitionMatrix` a la predicción en el filtro de Kalman?

Pregunta D.2

¿Cuál es la diferencia entre `measurementMatrix` y `transitionMatrix` en el contexto del seguimiento de objetos?